example C code

```
for (i=0; i<N; i++) A[i]++;
```

corresponding RISC-V assembly language (using s0 for i, s1 for N, and t0 as a pointer for stepping through the array):

```
A:      .space 40       # this goes in .bss section
                        # assuming here an array of 10 integers
                        # "A" can be used in our program as a
                        # name for starting memory address


# code follows, but there is a bug we need to fix … let's find it …

        mv s0, zero     # register zero (x0) permanently stores 0 (!)
        la t0, A        # t0 now has address of first word of array
loop:   beq s0, s1, next
        lw t1, 0(t0)
        addi t1, t1, 1
        sw t1, 0(t0)
        addi s0, s0, 1
        addi t0, t0, 1
        j loop
next:
```

an example RISC-V assembly language program using strings

(but there are some bugs we need to fix … *let's find them* …)

```
                .equ SYS_exit, 93
                .equ SYS_printStr, 248
                .equ SYS_readStr, 249

                .section .rodata
yes:            .asciz "Identical\n"
no:             .asciz "Not identical\n"
                .section .bss
str1:           .space 128
str2:           .space 128

                .section .text
                .globl _start
_start:         la a0, str1             # read in two strings from user
                li a7, SYS_readStr
                ecall
                la a0, str2
                li a7, SYS_readStr
                ecall
                la s0, str1             # get starting addresses of strings
                la s1, str2
cmpchar:        lw s2, 0(s0)
                lw s3, 0(s1)
                bne s2, s3, noteq
                addi s0, s0, 1
                addi s1, s1, 1
                bne s2, zero, cmpchar
                la a0, yes
                li a7, SYS_printStr
                ecall

noteq:          la a0, no
                li a7, SYS_printStr
                ecall
done:           li a0, 0                # exit(0)
                li a7, SYS_exit
                ecall
```

branch instructions

in addition to “beq” and “bne”, have some other branch instructions: “blt”, “bltu”, “bge”, “bgeu”

- the “u” suffix indicates that register contents should be interpreted as *unsigned* integers when do comparison

there are also branch pseudoinstructions

- “bgt”, “bgtu”, “ble”, “bleu”
- “beqz”, “bnez”, “bgtz”, “bgez”, “bltz”, blez” for comparisons against zero, e.g. “bnez s1, loop”

a pseudoinstruction does not have a directly corresponding machine language instruction; instead, assembler must implement using one or two real instructions that achieve the desired result

- for “bgt”, “bgtu”, “ble”, “bleu”, just need to reverse operands and use “blt”, “bltu”, “bge”, “bgeu”, respectively
- how about for “beqz”, “bnez”, … ?
- other pseudoinstructions include “mv” (how could this one be implemented?), “li”, “j”, “jr” (address to go to is in a register rather than given as a label), and “la”! (we’ll see later how these last four can be implemented …)
- also a convenient pseudoinstruction form of “lw/lb” (e.g., “lw s0, A”) … but “sw/sb” not so simple … *why?*

One more example of implementing control flow constructs: jmptabl example program on course Canvas site (we'll just look now at the parts in red, but you should download it and run it, and go through it instruction-by-instruction so that you fully understand it)

```
# function numbers for environment calls
        .equ SYS_exit, 93
        .equ SYS_readInt, 245
        .equ SYS_printStr, 248
        .equ SYS_readStr, 249

# ascii character codes
        .equ NEWLINE, 10
        .equ BLANK, 32

# read-only data section
         .section .rodata
prmpt1: .asciz "Enter a four character string:\n"
prmpt2: .asciz "Enter an integer between 0 and 3: "
newl:    .asciz "\n"
jmptab: .word A
        .word B
        .word C
        .word D

# uninitialized data section
        .section .bss
istr:   .space 128
ostr:   .space 6
junk:   .space 128

# code section
        .section .text
        .globl _start
```

```
_start:  la t0, ostr
         li t1, NEWLINE
         sb t1, 4(t0)
         sb zero, 5(t0)

again:   la a0, prmpt1
         li a7, SYS_printStr
         ecall

         la a0, istr
         li a1, 128
         li a7, SYS_readStr
         ecall

         la s0, istr             # part up to getint needed for length !=4 chars
         li s1, NEWLINE
         li s2, BLANK
         addi s3, s0, 4
         sb zero, 5(s0)

loop:    lbu s4, 0(s0)
         beq s1, s4, pad         # found a newline character in input string
         beq s0, s3, getint      # input length > 4
         addi s0, s0, 1
         j loop

pad:     beq s0, s3, getint
         sb s2, 0(s0)            # pad with blanks if string < 4 characters
         addi s0, s0, 1
         j pad

getint:  la a0, prmpt2
         li a7, SYS_printStr
         ecall

         li a7, SYS_readInt
```

```
        ecall
        mv s0, a0

        la a0, junk             # get rid of anything else on line that
        li a1, 128              #    integer was read on
        li a7, SYS_readStr
        ecall

        li t0, 3
        bgtu s0, t0, getint

        la s1, istr
        la s2, ostr

        la s3, jmptab
        add s0, s0, s0          # instead of the two "add s0, s0, s0"
        add s0, s0, s0          # instructions, could use "slli s0, s0, 2"
        add s3, s3, s0
        lw s3, 0(s3)
        jr s3

A:      lbu t1, 0(s1)
        sb t1, 0(s2)
        lbu t1, 1(s1)
        sb t1, 1(s2)
        lbu t1, 2(s1)
        sb t1, 2(s2)
        lbu t1, 3(s1)
        sb t1, 3(s2)
        j prnt

B:      lbu t1, 1(s1)
        sb t1, 0(s2)
        lbu t1, 2(s1)
        sb t1, 1(s2)
        lbu t1, 3(s1)
        sb t1, 2(s2)
```

```
        lbu t1, 0(s1)
        sb t1, 3(s2)
        j prnt

C:      lbu t1, 2(s1)
        sb t1, 0(s2)
        lbu t1, 3(s1)
        sb t1, 1(s2)
        lbu t1, 0(s1)
        sb t1, 2(s2)
        lbu t1, 1(s1)
        sb t1, 3(s2)
        j prnt

D:      lbu t1, 3(s1)
        sb t1, 0(s2)
        lbu t1, 0(s1)
        sb t1, 1(s2)
        lbu t1, 1(s1)
        sb t1, 2(s2)
        lbu t1, 2(s1)
        sb t1, 3(s2)

prnt:   la a0, ostr
        li a7, SYS_printStr
        ecall
        j again
```